

Lucrarea nr. 3: **Studiul circuitelor combinaționale de complexitate medie**

1. Scopul lucrării

În prima parte a lucrării se face o prezentare sintetică a principalelor circuite combinaționale de complexitate medie: decodificatorul, demultiplexorul, multiplexorul. Pentru fiecare tip de circuit se pune accent pe aspecte precum: structura internă, tabel de adevăr, particularități în funcționare.

În partea a doua se arată modul de utilizare a limbajului VHDL pentru descrierea acestor circuite în vederea implementării lor în structuri de tip CPLD sau FPGA.

2. Considerente teoretice

O posibilă clasificare a circuitelor integrate digitale, din punctul de vedere al gradului de integrare, este prezentată în tabelul 1.

Tabelul 1

Gradul de integrare	Numărul de porți	Circuite combinaționale (CLC)	Circuite secvențiale
MIC (Small Scale Integration)	≤ 12	- porți logice elementare;	- bistabili; - latch-uri;
MEDIU (Medium Scale Integration)	$12 \div 100$	- codificatoare , decodificatoare; - multiplexoare , demultiplexoare; - sumatoare; - comparatoare; - etc.	- registre; - numărătoare;
MARE (Large Scale Integration)	>100	- memorii fixe (ROM); - matrici logice programabile (PLA, PLD);	- registre mari; - memorii RAM;
FOARTE MARE (Very Large Scale Integration)	>1000	- structuri programabile de tip CPLD; - structuri programabile de tip FPGA;	- memorii RAM;

În proiectarea sistemelor digitale moderne, datorită apariției circuitelor realizate în tehnologie VLSI, tot mai des se pune problema realizării de sisteme pe un singur chip (**SoC**, System on Chip). În acest context, studiul circuitelor combinaționale de complexitate medie pare nejustificat. Această părere este greșită deoarece toate circuitele de complexitate medie se regăsesc ca părți integrante sau ca blocuri funcționale în structura circuitelor VLSI. Așadar, cunoașterea foarte exactă a tuturor facilităților oferite de aceste circuite, ne permite fie înțelegerea funcționării unui sistem complex realizat în tehnologie VLSI, fie proiectarea eficientă a unui astfel de sistem.

2. 1. Decodificatorul (DCD)

Decodificatorul este un circuit combinațional prevăzut cu n intrări (denumite cel mai adesea **intrări de selecție**) și 2^n ieșiri. Circuitul are proprietatea de a **recunoaște o combinație binară** aplicată pe intrările de selecție **prin activarea unei singure ieșiri**.

Numărul combinațiilor binare distincte (altfel spus, numărul de coduri) ce pot fi recunoscute de către un DCD este dependent de numărul intrărilor de selecție, iar numărul combinațiilor ce sunt semnalizate depinde de numărul de ieșiri disponibile.

În figura 1 se prezintă simbolul, schema logică de principiu și tabelul de adevăr pentru un decodor cu 8 ieșiri active pe zero logic. Așadar, recunoașterea unui cod se face prin trecerea în zero logic a unei singure ieșiri (cea asociată codului respectiv) iar restul ieșirilor se mențin în starea lor inactivă (în cazul de față unu logic). Facem precizarea că există și circuite DCD care au ieșirile active pe unu logic, schema lor internă fiind similară celei din figura 1.

Pentru a face distincție între intrările/ieșirile active pe unu și cele active pe zero, prin convenție internațională, intrările/ieșirile active pe zero sunt însoțite de un ceruleț în schema logică iar denumirea intrărilor/ieșirilor este însoțită de o bară similară celei de negație ($\bar{Y}_{out}$).

Printre circuitele uzuale disponibile pe piață putem enumera:

- decodoare BCD/zecimal: 7442, 7445, 74141, 74145, aceste circuite prezintă patru intrări de selecție și numai 10 ieșiri (active în "zero logic") din cele 16 posibile;

decodoare BCD/7segmente: 7446, 7447 circuite utilizate pentru comanda afișajelor numerice;

2. 2. Demultiplexorul (DMUX)

Acest circuit poate fi privit ca un DCD prevăzut cu o intrare suplimentară de validare a funcționării circuitului. În consecință un DMUX prezintă: n intrări de selecție, o intrare de validare și 2^n ieșiri.

Intrarea de validare, denumită de regulă *Enable*, controlează funcționarea DMUX. Pentru cazul unui DMUX cu intrarea de validare activă pe zero logic, așa cum este cazul celui prezentat în figura 2, putem avea următoarele situații:

- dacă intrarea de validare este activată, $\bar{E} = 0$, funcționarea demultiplexorului este identică cu a unui DCD (în funcție de codul aplicat pe intrările de selecție se activează doar o singură ieșire);
- dacă intrarea de validare este neactivată, $\bar{E} = 1$, toate ieșirile sunt forțate în starea lor inactivă, indiferent de codul binar aplicat pe intrările de selecție.

La o primă vedere, se poate spune că DMUX este un DCD mai flexibil deoarece prin intermediul intrării de validare se poate controla momentele de timp în care DMUX are voie să funcționeze.

Principala funcție a unui DMUX este aceea de a distribui informația digitală prezentă pe intrarea de validare $\bar{E}$, spre una din liniile de ieșire. Este evident că selectarea liniei de ieșire se face prin intermediul intrărilor de selecție. **Deci, din punct de vedere funcțional, un DMUX poate fi asemănat cu un comutator rotativ.**

Pentru a demonstra această ultimă proprietate a unui DMUX, ce nu este evidentă la prima vedere, să considerăm că pe intrările de selecție ale circuitului din figura 2, aplicăm combinația binară $BA=01$, ceea ce înseamnă că ieșirea aleasă este $\bar{1}$. În funcție de starea logică a intrării $\bar{E}$ pot exista două situații:

- dacă $\bar{E} = 0$, circuitul DMUX lucrează ca un DCD și, datorită codului aplicat intrărilor de selecție, se activează ieșirea $\bar{1}$ (trece în starea "0") în timp ce restul ieșirilor rămân inactive (starea logică "1");
- dacă $\bar{E} = 1$, DMUX nu este validat, deci toate ieșirile sunt forțate în starea inactivă (starea logică "1").

Din analiza celor două situații se observă că starea logică a ieșirii selectate, în cazul de față $\bar{1}$, este identică cu starea logică a intrării de validare $\bar{E}$. Cu alte cuvinte putem afirma că **informația digitală prezentă pe intrarea $\bar{E}$ este transmisă (direcționată) spre o ieșire indicată de codul aplicat intrărilor de selecție.**

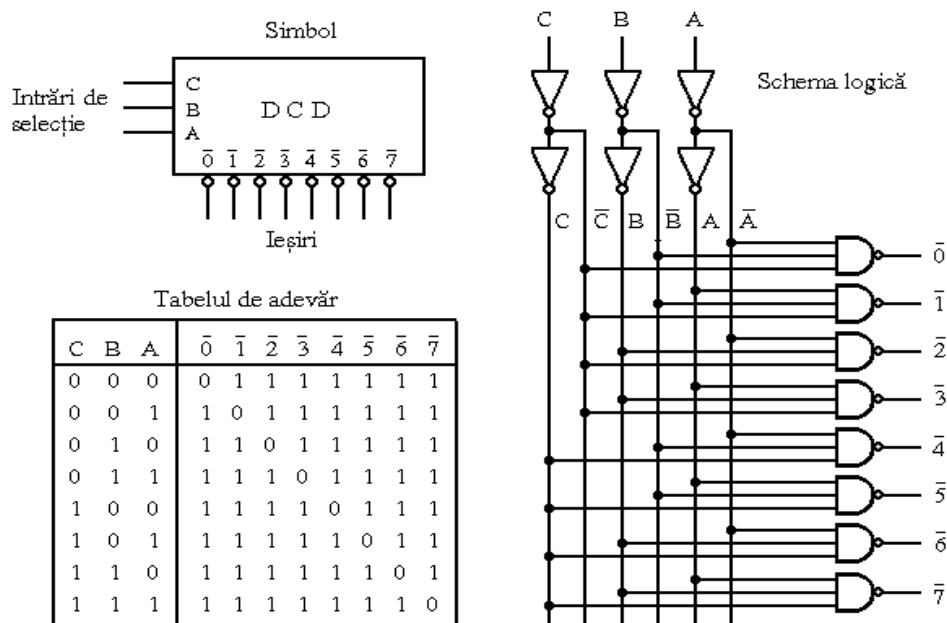


Fig.1. Schema logică, tabelul de adevăr și simbolul unui decodificator cu 8 ieșiri active pe zero logic

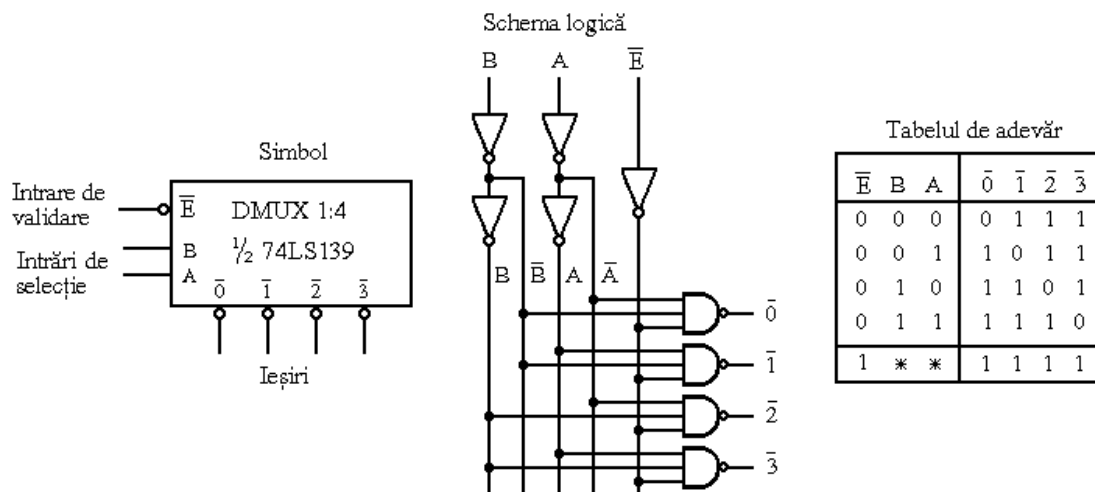


Fig.2. Schema logică, simbolul și tabelul de adevăr pentru DMUX 1:4 de tip 74LS139

Un circuit DMUX foarte des utilizat este circuitul 74LS138, care prezintă 8 ieșiri de date active pe zero logic și trei intrări de validare, două active pe zero logic iar a treia pe unu logic. Activarea unei ieșiri pentru acest circuit se poate face numai dacă simultan sunt îndeplinite condițiile: $G1A=1$, $G2A=0$ și $G2B=0$. Schema logică și tabelul de adevăr sunt prezentate în figura 3.

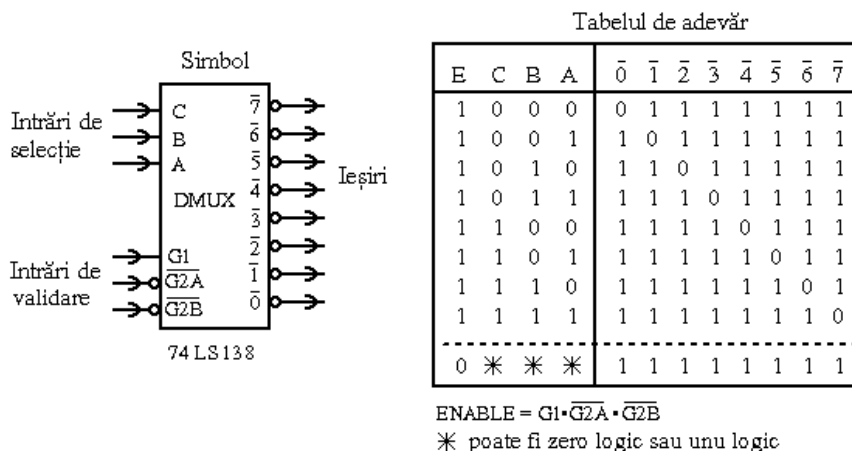


Fig.3. Schema logică și tabelul de adevăr pentru circuitul demultiplexor 3:8 de tip 74LS138

2.3. Circuitul multiplexor (MUX)

Multiplexorul este un circuit logic combinațional prevăzut cu: n intrări de selecție, 2^n intrări de date și o singură ieșire de date. Prin intermediul unui cuvânt de cod de n biți (adresa de selecție), multiplexorul conectează la ieșirea de date una din intrările sale. Funcționarea acestui circuit poate fi asemănată cu cea a unui comutator rotativ cu mai multe poziții de intrare, așa cum se prezintă în figura 4.b. Intrarea de validare $\overline{E}$, permite validarea funcționării ($\overline{E}=0$) sau blocarea funcționării ($\overline{E}=1$) circuitului.

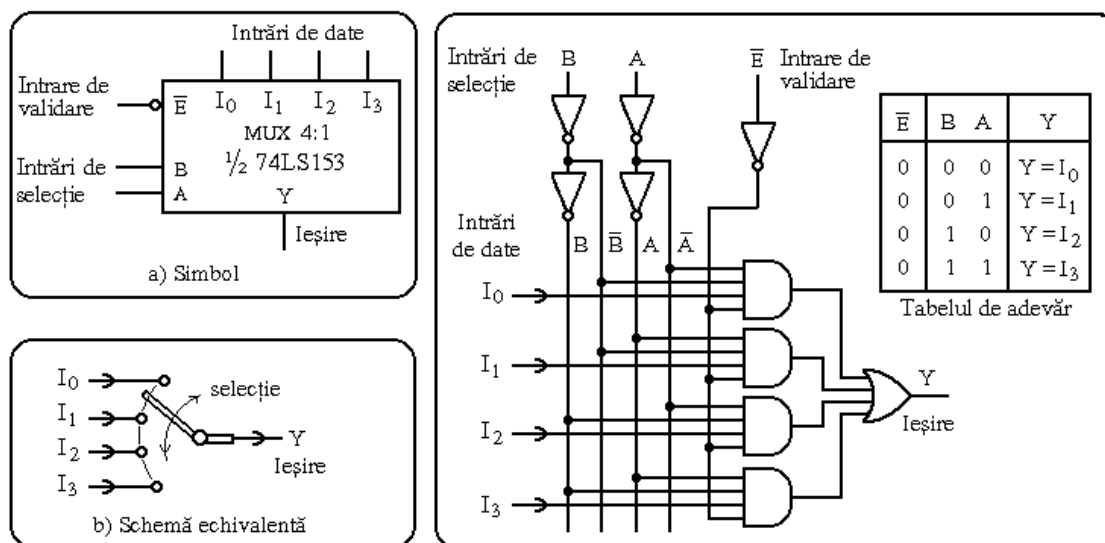


Fig. 4. Schema logică, tabelul de adevăr și simbolul pentru un MUX 4:1 de tip 74LS153

2.4. Utilizarea limbajului VHDL pentru descrierea circuitelor combinaționale de complexitate medie.

După cum este deja cunoscut, limbajul VHDL permite o descriere foarte ușoară a funcționării unui circuit/sistem logic. Reamintim că limbajul VHDL, permite descrierea circuitelor/sistemelor logice în câteva moduri:

- **descriere structurală** - proiectantul impune schema logică iar sarcina mediului software este de a "amplasa" corect această schemă în resursele interne ale circuitului în care se dezvoltă aplicația;
- **descriere comportamentală** - proiectantul face o descriere de nivel înalt a circuitului/sistemului urmând ca sinteza propriu-zisă a schemei logice să rămână în sarcina mediului software de dezvoltare.
- **Metode combinate** - proiectantul poate să opteze pentru descriere structurală pentru anumite blocuri funcționale și descriere comportamentală pentru altele.

De regulă, limbajul VHDL este conceput pentru implementarea în paralel a circuitelor logice, de aceea, de cele mai multe ori, ordinea introducerii declarațiilor nu este importantă. Singura modalitate permisă de VHDL de a introduce operații cu execuție secvențială (una după alta, în ordinea scrierii lor în program) constă în folosirea declarației de tip **process**. *Atenție, de această dată, ordinea declarațiilor din corpul unui proces are importanță !*

♦ **Exemplul 1:** Descrierea în limbaj VHDL a unui MUX 4:1 (Implementare concurrentă – varianta 1)

Observații	Codul VHDL
Secțiune dedicată includerii de librării	<pre>library IEEE; use IEEE.std_logic_1164.all;</pre>
Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: mux4 - intrările de selecție sunt: s1 și s0 ; - intrările de date sunt: d0, d1, d2, d3 ; - ieșirea de date este notată Yout ;	<pre>entity mux4 is port (d0, d1, d2, d3 : in std_logic; s1, s0 : in std_logic; yout : out std_logic); end mux4;</pre>
Secțiune dedicată descrierii arhitecturii. În cazul de față: - numele arhitecturii este: abc_arh ; - arhitectura este asociată cu entitatea: mux4 ; - descrierea funcționării se face cu două declarații concurente: una calculează semnalul intern sel , iar cealaltă calculează ieșirea yout ; - la prima vedere descrierea pare greșită deoarece semnalul intern sel , este folosit înainte de a fi calculat; - descriere este corectă deoarece succesiunea declarațiilor concurente nu contează, ele sunt parcurse în paralel; - semnalul sel , este recunoscut numai în interiorul arhitecturii abc_arh ;	<pre>architecture abc_arh of mux4 is signal sel: integer begin with sel select yout <= d0 when 0, d1 when 1, d2 when 2, d3 when 3, 'X' when others; sel <= 0 when s0='0' and s1='0' else 1 when s0='1' and s1='0' else 2 when s0='0' and s1='1' else 3 when s0='1' and s1='1' else 4; end abc_arh;</pre>

♦ **Exemplul 2:** Descrierea în limbaj VHDL a unui MUX 4:1 (Implementare concurrentă – variantă greșită)

Observații	Codul VHDL
Secțiune dedicată includerii de librării	<pre>library IEEE; use IEEE.std_logic_1164.all;</pre>
Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: mux4 - intrările de selecție sunt: s1 și s0 ; - intrările de date sunt: d0, d1, d2, d3 ; - ieșirea de date este notată yout ;	<pre>entity mux4 is port (d0, d1, d2, d3 : in std_logic; s1, s0 : in std_logic; yout : out std_logic); end mux4;</pre>
Secțiune dedicată descrierii arhitecturii. În cazul de față: - descrierea se face cu 4 declarații concurente, la prima vedere pare corectă dar în realitate este greșită ; Unde este greșeala? - fiecare atribuire pentru yout , generează un nou driver pentru comanda semnalului de ieșire yout ; - așadar avem 4 drivere, fiecare încearcă să impună propria sa valoare logică asupra firului de ieșire yout , lucru ce conduce la conflict; - spre exemplu pentru s1s0=10 , un driver forțează starea lui d2 , iar celelalte forțează starea zero logic;	<pre>architecture gresit of mux4 is begin yout <= d0 when s0='0' and s1='0' else '0'; yout <= d1 when s0='1' and s1='0' else '0'; yout <= d2 when s0='0' and s1='1' else '0'; yout <= d3 when s0='1' and s1='1' else '0'; end gresit;</pre>
Cum se poate corecta greșeala? - varianta corectă trebuie să folosească o singură atribuire pentru yout ;	<pre>architecture corect of mux4 is begin yout <= d0 when s0='0' and s1='0' else, d1 when s0='1' and s1='0' else, d2 when s0='0' and s1='1' else, d3 when s0='1' and s1='1' else, 'X'; -- pentru necunoscut end corect;</pre>

♦ **Exemplul 3:** Descrierea în limbaj VHDL a unui MUX 4:1 (Implementare secvențială - varianta 1)

Observații	Codul VHDL
Secțiune dedicată includerii de librării	<pre>library IEEE;</pre>

Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: <i>mux4</i> - vectorul de selecție cu două componente: <i>sel</i>; - intrările de date sunt: <i>d0, d1, d2, d3</i>; - ieșirea de date este notată <i>Yout</i>;	<pre>use IEEE.std_logic_1164.all; entity mux4 is port (d0, d1, d2, d3 : in std_logic; sel:in std_logic_vector(1 downto 0); Yout : out std_logic); end mux4;</pre>
Secțiune dedicată descrierii arhitecturii. În cazul de față: - descrierea funcționării se face cu un proces ce are ca listă de senzitivități toate intrările de date ale MUX; - procesul este parcurs pentru orice modificare de stare logică apărută pe oricare intrare de date; - declarația <i>case</i> poate fi folosită numai în interiorul unui proces; Observații referitoare la procese: - Un proces este parcurs pentru orice schimbare de stare logică apărută la oricare variabilă din lista de senzitivități; - Declarațiile din corpul procesului sunt parcurse una după alta (execuție secvențială) și nu în paralel;	<pre>architecture arh_3 of mux4 is begin P1: process (d0, d1, d2, d3) begin case sel is when "00" => Yout <= d0; when "01" => Yout <= d1; when "10" => Yout <= d2; when others => Yout <= d3; end case; end process P1; end arh_3;</pre>

◆ **Exemplul 4:** Descrierea în limbaj VHDL a unui MUX4:1 (Implementare secvențială - varianta 2)

Observații	Codul VHDL
Secțiune dedicată includerii de librării	<pre>library IEEE; use IEEE.std_logic_1164.all;</pre>
Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: <i>mux4_1</i>; - selecția se face cu un vector pe 2 biți: <i>adr</i>; - datele de intrare: <i>d0, d1, d2, d3</i>; - ieșirea de date: <i>data_out</i>;	<pre>entity mux4_1 is port (adr : in std_logic_vector(1 downto 0); d0,d1,d2,d3 : in std_logic; data_out : in std_logic); end mux4_1;</pre>
Secțiune dedicată descrierii arhitecturii. În cazul de față: - numele arhitecturii este: <i>arch_1</i>; - arhitectura se asociază cu entitatea <i>mux4_1</i>; - procesul <i>P1</i>, este sensibil la intrarea de selecție și la intrările de date; - în funcție de combinația de pe intrările de selecție, ieșirea <i>data_out</i> are o singură atribuire;	<pre>architecture arch4 of mux4_1 is begin P1: process (adr, data_in) begin case adr is when "00" => data_out <= d0; when "01" => data_out <= d1; when "10" => data_out <= d2; when "11" => data_out <= d3; when others => data_out <= "--"; end case; end process P1; end arch4;</pre>

◆ **Exemplul 5:** Descrierea în limbaj VHDL a unui DMUX 1:8 ((Implementare secvențială)

Observații	Codul VHDL
Secțiune dedicată includerii de librării	<pre>library IEEE; use IEEE.std_logic_1164.all;</pre>
Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: <i>dmux_A</i> - intrările de selecție sunt introduse prin: <i>code_in</i>; - ieșirea de date este notată <i>Yout</i>; - intrarea de validare este declarată: <i>En</i>.	<pre>entity dmux_A is port (code_in :in std_logic_vector(2 downto 0); En:in std_logic; Yout:out std_logic_vector(7 downto 0)); end dmux_A;</pre>
Secțiune dedicată descrierii arhitecturii. Observații referitoare la procese: - Un proces este parcurs pentru orice schimbare de stare logică apărută la oricare variabilă din lista de senzitivități; - Declarațiile din corpul procesului sunt parcurse una după alta (execuție secvențială) și nu în paralel; În cazul de față: - descrierea funcționării se face cu procesul P1, ce are în lista de senzitivități intrarea <i>code_in</i>; - procesul este parcurs la fiecare modificare a codului aplicat pe intrarea <i>code_in</i>;	<pre>architecture dmux_arh of dmux_A is begin P1: process (code_in) begin if (En='0') then Yout <= (others => '0'); else case code_in is when "000" => Yout <= "00000001"; when "001" => Yout <= "00000010"; when "010" => Yout <= "00000100"; when "011" => Yout <= "00001000"; when "100" => Yout <= "00010000";</pre>

-intrarea de validare și ieșirile sunt active pe unu logic; -declarația <i>case</i> este parcursă doar dacă E=1;	<pre> when "101" => Yout <= "00100000"; when "110" => Yout <= "01000000"; when "111" => Yout <= "10000000"; when others => Yout <= "00000000"; end case; end if; end process; end dmux_arh; </pre>
---	--

◆ **Exemplul 6:** Descrierea în limbaj VHDL a unui decodificator zecimal cu ieșiri active pe unu logic

Observații	Codul VHDL
Secțiune dedicată includerii de librării	<pre> library IEEE; use IEEE.std_logic_1164.all; </pre>
Secțiune dedicată descrierii entității. Obs.: elementele unui vector pot fi declarate în ordine descrescătoare (cazul intrării <i>bcd</i>) sau în ordine crescătoare (cazul ieșirii <i>zec</i>).	<pre> entity bcd_zec is port (bcd : in std_logic_vector(3 downto 0); zec : out std_logic_vector(0 to 9)); end bcd_zec; </pre>
Secțiune dedicată descrierii arhitecturii. În cazul de față: -numele arhitecturii este: <i>dec_arh</i>; -arhitectura se asociază cu entitatea <i>bcd_zec</i>; -procesul <i>P1</i>, este sensibil la codul de intrare <i>bcd</i>; -descrierea este corectă deoarece se face o singură atribuire pentru ieșirea <i>zec</i>; -ieșirile sunt active pe unu logic;	<pre> architecture dec_arh of bcd_zec is begin P1: process (bcd) begin case bcd is when "0000" => zec <= "1000000000"; when "0001" => zec <= "0100000000"; when "0010" => zec <= "0010000000"; when "0011" => zec <= "0001000000"; when "0100" => zec <= "0000100000"; when "0101" => zec <= "0000010000"; when "0110" => zec <= "0000001000"; when "0111" => zec <= "0000000100"; when "1000" => zec <= "0000000010"; when "1001" => zec <= "0000000001"; when others => zec <= "0000000000"; end case; end process P1; end dec_arh; </pre>

◆ **Exemplul 7:** Sumator pe 2 biți cu afișarea rezultatului pe un digit cu 7 segmente

În acest exemplu se descrie un CLC care realizează sumarea a doi operanzi, fiecare fiind exprimat pe 2 biți. Rezultatul se afișează pe un digit cu 7 segmente. Deoarece cel mai mare număr pe 2 biți este 3, rezultatul maxim al operației de adunare este 6, deci poate fi afișat pe un singur digit.

Observații	Codul VHDL
Secțiune dedicată includerii de librării	<pre> library IEEE; use IEEE.std_logic_1164.all; use IEEE.std_logic_unsigned.all; use IEEE.std_logic_arith.all; </pre>
Secțiune dedicată descrierii entității. În cazul de față: -avem două intrări de 2 biți: <i>x</i>, <i>y</i>; -o ieșire pe 7 biți pentru comanda segmentelor afișajului <i>afis</i>; -o ieșire pentru comanda catodului comun al afișajului <i>d_afis</i>;	<pre> entity sum2 is port(d_activare: out std_logic; x,y:in std_logic_vector(1 downto 0); afis:out std_logic_vector(6 downto 0)); end sum2; </pre>
Secțiune dedicată descrierii arhitecturii. În cazul de față: -numele arhitecturii este: <i>arh_sum2</i>; -asocierea arhitecturii se face cu entitatea <i>sum2</i>; -descrierea folosește numai declarații concurente (pentru operația de sumare, pentru activare afișaj, pentru decodificare BCD-7seg.); Observații: - o declarație concurentă este executată pentru orice modificare de stare logică a semnalelor implicate în ea, deci <i>sum</i> este reactualizat pentru orice modificare apărută pe <i>x</i>	<pre> architecture arh_sum2 of sum2 is signal suma : std_logic_vector(3 downto 0); begin suma <= x + y; d_activare <= '1'; with suma select afis<= "1000000" when "0000", --0 "1111001" when "0001", --1 "0100100" when "0010", --2 "0110000" when "0011", --3 "0011001" when "0100", --4 "0010010" when "0101", --5 </pre>

sau pe y ;	<pre> "0000010" when "0110", --6 "1111000" when "0111", --7 "0000000" when "1000", --8 "0010000" when "1001", --9 "1111111" when others; --alte situatii end arh sum2;</pre>
Fisierul de constrângeri - rezultatul se afișează pe primul digit al machetei (prima linie din fișierul de constrângeri); - următoarele 7 linii sunt pentru comanda segmentelor digitului; - introducerea operandului x se face prin SW1 și SW2; - introducerea operandului y se face prin SW7 și SW8;	<pre> NET "d_activare" LOC = "P70"; NET "afis<0>" LOC = "P39"; NET "afis<1>" LOC = "P41"; NET "afis<2>" LOC = "P44"; NET "afis<3>" LOC = "P46"; NET "afis<4>" LOC = "P48"; NET "afis<5>" LOC = "P51"; NET "afis<6>" LOC = "P53"; NET "x<1>" LOC = "P37"; NET "x<0>" LOC = "P40"; NET "y<1>" LOC = "P52"; NET "y<0>" LOC = "P54";</pre>



3. Desfășurarea lucrării

3.1. Studiul circuitelor decodificatoare (DCD).

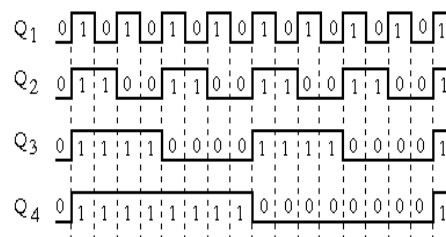
A. Referitor la circuitul decodificator din figura 1, răspundeți la următoarele întrebări:

- Ce formă canonică a fost folosită în proiectarea sa ?
- Se poate modifica schema logică pentru a obține un circuit cu ieșirile active pe 1 logic ? Cum ?
- Cât timp se menține activă o ieșire ?
- Dacă $C=0$, $B=0$ și $A=1$, codul perceput de circuit este 001 sau 100? Ce ieșire se activează în acest caz?
- Ce succesiune de coduri trebuie aplicate pe intrările de selecție dacă dorim activarea ieșirilor în ordinea: $\overline{3}, \overline{5}, \overline{7}, \overline{9}$?

B. Pentru un circuit decodificator BCD-zecimal, spre exemplu circuitul 74LS442, se cere desenarea corelată în timp a semnalelor de ieșire dacă pe intrările de selecție sunt aplicate semnalele din figura alăturată în ordinea:

- a) $Q_1 \rightarrow A$, $Q_2 \rightarrow B$, $Q_3 \rightarrow C$, $Q_4 \rightarrow D$;
b) $Q_1 \rightarrow D$, $Q_2 \rightarrow C$, $Q_3 \rightarrow B$, $Q_4 \rightarrow A$;

Cum explicați diferențele apărute ?



C. Scrieți tabelul de adevăr pentru un decodificator cu 4 intrări și 7 ieșiri destinat comenzii unui afișaj numeric. Intrările sunt notate prin $DCBA$, iar ieșirile $a b c d e f g$. Ieșirile sunt active pe unu logic (unu logic = segment aprins). Tabelul trebuie realizat astfel încât cifrele să arate ca în figura alăturată.



3.2. Studiul circuitelor demultiplexoare (DMUX).

A. Referitor la circuitul decodificator din figura 2, răspundeți la următoarele întrebări:

- Se poate modifica schema logică pentru a obține un circuit cu ieșirile active pe 1 logic? Cum ?
- Cât timp se menține activă o ieșire ?
- Dacă $\overline{E} = 0$, $B=0$ și $A=1$, codul perceput de circuit este 01 sau 10? Ce ieșire se activează în acest caz?
- Care sunt cele două moduri în care se poate face dezactivarea unei ieșiri ?
- Ce succesiune de coduri trebuie aplicate pe intrările de selecție dacă dorim activarea ieșirilor în ordinea: $\overline{1}, \overline{3}, \overline{2}, \overline{0}$?

B. Care este modul de conectare al unui decodificator zecimal, spre exemplu circuitul 74LS442, pentru a obține o funcționare similară unui DUX 1:8 ?

3.3. Studiul circuitelor multiplexoare (MUX).

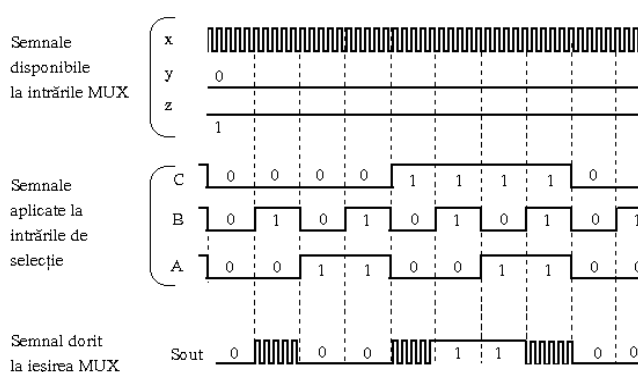
A. Referitor la circuitul decodificator din figura 5, răspundeți la următoarele întrebări:

- În ce stare logică se află ieșirea de date dacă $\overline{E} = 1$?
- Cât timp se menține activă o ieșire ?
- Dacă $\overline{E} = 0$, $B=0$ și $A=1$, codul perceput de circuit este 01 sau 10? Ce intrare este conectată la ieșirea de date ?

- B.** În ce ordine trebuie conectate semnalele x, y, z la intrările unui MUX8:1 astfel încât la ieșire să obținem semnalul *Sout*?

Modul de comandă a selecțiilor și forma semnalelor x, y, z se prezintă în figura alăturată.

Semnalul *Sout* poate fi obținut prin utilizarea unui MUX 4:1? Dacă da, care este schema de conectare?



3.4. Utilizarea ISE WebPack pentru descrierea aplicațiilor sub formă de scheme logice

- A.** Folosind facilitatea mediului de dezvoltare ISE WebPack, prin care se permite descrierea sistemelor digitale prin intermediul schemelor logice, se cere:
- Implementarea și verificarea pe macheta de laborator a schemei de DCD din figura 1.
 - Implementarea și verificarea pe macheta de laborator a schemei de DMUX din figura 2.
 - Implementarea și verificarea pe macheta de laborator a schemei MUX din figura 4.
- B.** Folosind facilitatea mediului de dezvoltare ISE WebPack, prin care se permite descrierea sistemelor digitale prin intermediul schemelor logice, se cere:
- Implementarea și verificarea pe macheta de laborator a schemei de DCD din figura 1.
 - Implementarea și verificarea pe macheta de laborator a schemei de DMUX din figura 2.
 - Implementarea și verificarea pe macheta de laborator a schemei MUX din figura 4.

Mod de lucru: Se folosesc indicațiile de la sfârșitul lucrării.

3.5. Implementarea circuitelor logice cu ajutorul limbajului VHDL

- A.** Ținând cont de descrierile VHDL prezentate în exemplele 1÷6, se cere codul VHDL și implementarea pe macheta de laborator pentru:
- Un circuit de MUX8:1 cu intrare de validare activă pe zero logic;
 - Un decodificator zecimal cu ieșiri active pe zero logic;
 - Un DMUX 1:8. După implementare, verificați că starea logică a intrării de date selectate este transmisă și la ieșirea circuitului.
- B.** Implementați pe macheta de laborator cu CPLD, sumatorul pe 2 biți prezentat în exemplul 7.
- C.** Adaptați metoda de descriere a unui decodificator (exemplul 6), pentru implementarea decodificatorului BCD-7 segmente de la punctul 3.1.C. din desfășurarea lucrării.
- D.** Realizați o descriere în limbaj VHDL și implementați pe macheta cu CPLD, un circuit de transcodare pe 4 biți care să facă trecerea de la coul binar natural la coul Gray.

Mod de lucru: Se folosesc indicațiile de la sfârșitul lucrării.

4. Indicații privind modul de lucru

Pentru fiecare aplicație este necesară deschiderea unui nou proiect după metodologia prezentată într-o lucrare de laborator anterioară. Toate aplicațiile din această lucrare necesită doar un singur fișier sursă (fie schemă logică, fie sursă VHDL) și un singur fișier de constrângeri.

Referitor la conectarea intrărilor și a ieșirilor din circuit facem următoarele precizări:

- Intrările de selecție și cele de date se vor conecta la switch-urile (comutatoare cu două poziții) de pe macheta de laborator. În acest mod, trecerea comutatorului de pe o poziție pe alalta echivalează cu schimbarea stării logice a variabilei de intrare.
- Variabilele de ieșire se vor conecta la LED-urile de pe macheta de laborator. În acest mod, în momentul în care o variabilă de ieșire este în unu logic, LED-ul asociat luminează.
- În cazul aplicației de sumare fiecare număr de intrare respectiv ieșire va fi reprezentat printr-un vector. Un vector de intrare trebuie conectat la un pachet de switch-uri (un număr corespunzător de switch-uri ce sunt amplasate unul lângă altul) iar vectorul de ieșire va fi reprezentat pe un pachet de LED-uri.
- Fișierul de constrângeri pentru exemplul 6 este prezentat alăturat. Pentru introducerea numărului x s-au folosit primele trei comutatoare (SW1, SW2, SW3), iar pentru y s-au folosit ultimele trei comutatoare (SW6, SW7, SW8). Afișarea rezultatului se face pe primele 5 LED-uri (LD1÷LD5).

NET "x<0>" LOC = "P37";
NET "x<1>" LOC = "P40";
NET "x<2>" LOC = "P43";

NET "y<0>" LOC = "P54";
NET "y<1>" LOC = "P52";
NET "y<2>" LOC = "P50";

NET "z<0>" LOC = "P75";
NET "z<1>" LOC = "P71";
NET "z<2>" LOC = "P67";
NET "z<3>" LOC = "P65";
NET "z<4>" LOC = "P62";

